

FitBuddy

AI Fitness Plan Generator

Full Project Documentation

Item	Details
Project type	FastAPI web application with Jinja2, SQLite/SQLAlchemy, and Gemini integration.
Core purpose	Generate a structured 7-day fitness plan and a concise nutrition/recovery tip, then revise the plan from user feedback.
AI fallback	Built-in mock mode supports local demos and automated tests without a Gemini API key.
Source basis	Prepared from the supplied FitBuddy project ZIP archive and three supplied screenshots.

Scope note: Configuration values and model identifiers are documented as they appear in the supplied source. They are not independently verified in this document.

Table of Contents

1	— Project Overview
2	— Features and Requirements
3	— Architecture
4	— Technology Stack
5	— Project Structure
6	— Application Workflow
7	— UI Documentation
8	— AI Service Design
9	— Database and Persistence
10	— REST API Reference
11	— Configuration
12	— Installation and Running
13	— Testing
14	— Security and Safety
15	— Troubleshooting
16	— Limitations and Future Work
17	— Deployment Checklist
18	— Appendix: Source Reference

1. Project Overview

FitBuddy is a browser-based AI fitness planning application. A user supplies a compact fitness profile—name, user ID, age, weight, goal, intensity, and experience level—and the system generates a seven-day workout plan plus a concise nutrition or recovery tip. A feedback form then allows the user to request a revised plan.

The supplied implementation uses FastAPI for HTTP routing, Jinja2 for server-rendered HTML, SQLAlchemy with SQLite for persistence, and the Google GenAI Python SDK for AI generation. When a Gemini key is absent and mock mode is enabled, the application returns deterministic local demo content.

The project also exposes a JSON REST API under `/api/v1`, a Basic-Auth-protected admin dashboard, environment-based configuration, and automated tests.

Primary goals

- Simple browser workflow for personalized 7-day planning.
- Structured AI output that is easy to render and revise.
- Persistence of users and current plans.
- HTML interface plus programmatic REST API.
- Local mock-AI mode for development and tests.
- Prompt-level guardrails against unsafe exercise, restrictive dieting, and medical-treatment claims.

Observed implementation

- The supplied screenshots show profile entry, generated plan output, and feedback-updated output.
- The README describes the solution as FastAPI + Jinja2 + SQLite + Gemini.
- The source separates configuration, AI service, routes, schemas, and database responsibilities.
- The tests cover home page, health endpoint, plan creation, and feedback revision.

2. Features and Requirements

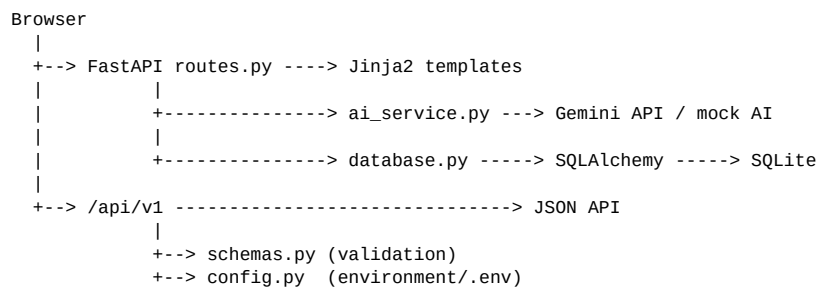
Feature	Description	Implementation
Fitness profile	Collect name, user ID, age, weight, goal, intensity, and experience.	HTML form + Pydantic validation
7-day plan	Exactly seven days with focus, warm-up, workout, cool-down, and notes.	ai_service.py
Nutrition/recovery tip	One concise general-wellness tip, prompted to be 80 words or fewer.	ai_service.py
Feedback revision	Revise the current plan from user feedback.	HTML + REST endpoints
Persistence	Store users and plans in SQLite.	SQLAlchemy database.py
REST API	Health, create, retrieve, and revise operations.	routes.py /api/v1
Admin dashboard	View stored users, plans, and feedback after Basic Auth.	/view-all-users
Mock AI	Deterministic plan generation without an external AI key.	mock_response()

Validation rules

- Age: 13–100.
- Weight: greater than 20 kg and less than 400 kg.
- Goals: weight loss, muscle gain, general wellness, flexibility, endurance.
- Intensity: low, medium, high.
- Experience: beginner, intermediate, advanced.
- Feedback: maximum 1,200 characters.

3. System Architecture

The system follows a small layered architecture: FastAPI routes coordinate validation, AI calls, persistence, and HTML/JSON responses.



Create-plan lifecycle

- POST /generate-workout receives the form.
- `UserInput` validates and normalizes input.
- `save_user()` creates or updates the user.
- Workout and nutrition AI functions generate content.
- `save_plan()` stores the plan and marks previous plans not current.
- `result.html` renders profile, plan, tip, and feedback form.

Feedback lifecycle

- Feedback is validated and the current plan is loaded.
- `update_workout_plan()` sends profile, plan, and feedback to the AI service.
- `update_plan()` stores updated_plan and feedback.
- `result.html` renders the revised plan.

4. Technology Stack

Layer	Technology	Role
Language	Python 3.11+ per README	Application code
Web framework	FastAPI	HTTP routing, dependency injection, API docs
Server	Uvicorn	ASGI server
Templates	Jinja2	Server-rendered HTML
Validation	Pydantic / pydantic-settings	Request and environment validation
Persistence	SQLAlchemy + SQLite	ORM and local database
AI	google-genai	Gemini API integration
Testing	pytest + httpx	Automated application/API tests
Frontend	Custom CSS	Responsive dark interface

Supplied dependency specification:

```
fastapi>=0.115,<1
uvicorn[standard]>=0.30,<1
jinja2>=3.1,<4
sqlalchemy>=2.0,<3
python-multipart>=0.0.9,<1
pydantic>=2.8,<3
pydantic-settings>=2.4,<3
google-genai>=1.0,<2
python-dotenv>=1.0,<2
pytest>=8,<9
httpx>=0.27,<1
```

5. Project Structure

```
fitbuddy_project/
├── app/
│   ├── ai_service.py
│   ├── config.py
│   ├── crud.py
│   ├── database.py
│   ├── main.py
│   ├── routes.py
│   └── schemas.py
├── static/css/styles.css
├── templates/
│   ├── all_users.html
│   ├── base.html
│   ├── error.html
│   ├── index.html
│   └── result.html
├── tests/test_app.py
├── .env.example
├── .gitignore
├── README.md
└── requirements.txt
```

Module responsibilities

- `main.py` — creates FastAPI app, initializes DB at startup, mounts static files, includes routers.
- `routes.py` — HTML routes, API endpoints, admin guard, AI orchestration, rendering.
- `schemas.py` — Pydantic input/response models and allowed-value validators.
- `database.py` — SQLAlchemy User/Plan models and persistence helpers.
- `ai_service.py` — prompts, Gemini calls, JSON cleaning, formatting, and mock responses.
- `config.py` — environment/.env settings.
- `crud.py` — compatibility re-exports of database helpers.
- `templates/` — shared layout, form, result, admin, and error pages.
- `styles.css` — responsive dark UI.
- `test_app.py` — automated functional/API tests.

6. Application Workflow

A. Profile entry

The home page presents a responsive form. Browser-side controls constrain common values, while the server performs authoritative Pydantic validation. The form posts to **/generate-workout**.

B. AI generation

The workout prompt requests JSON containing an overview and exactly seven day objects. Each day contains focus, warm-up, main workout, cool-down, and notes. The prompt asks for accessible exercises and recovery/rest while prohibiting unsafe challenges and medical treatment.

The nutrition prompt requests one general wellness tip of 80 words or fewer and avoids restrictive dieting, calorie targets, supplements, and medical treatment.

C. Storage

A user record is created or updated by user ID. When a new plan is saved, older plans for that user are marked not current and the new plan becomes current.

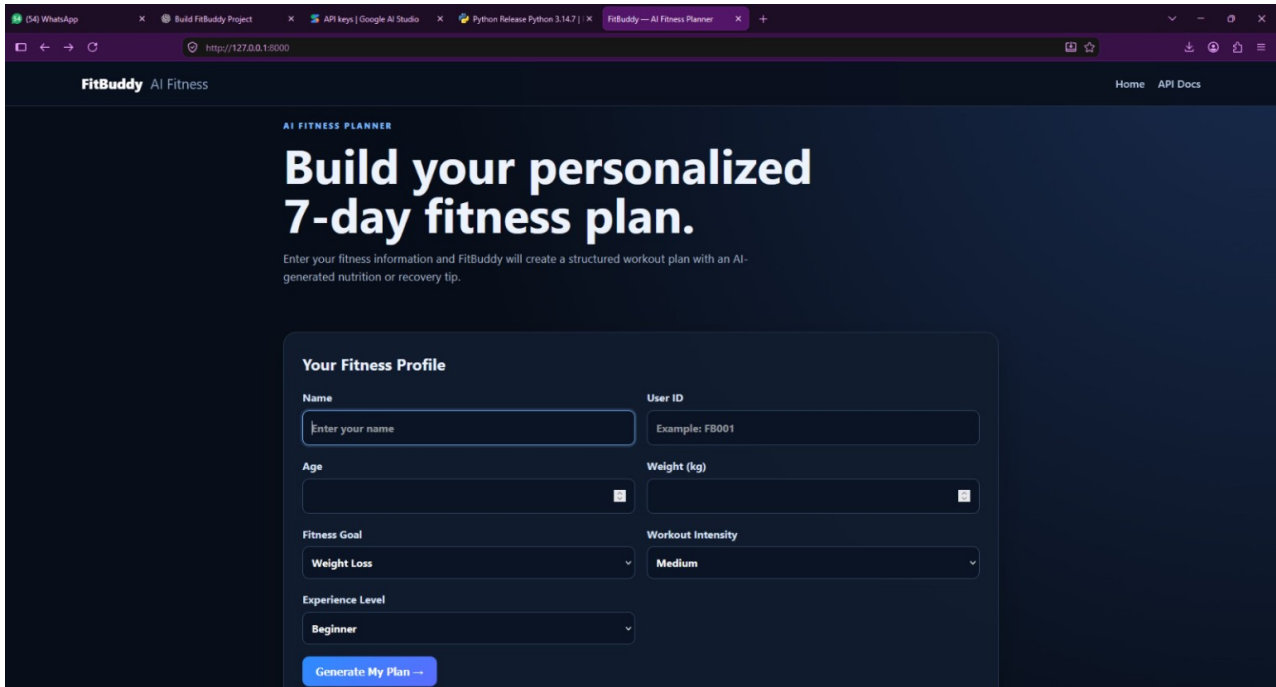
D. Feedback loop

The feedback form submits text and user ID. The service loads the current plan, asks the model to revise it while preserving useful elements and exactly seven days, then stores the revised plan in `updated_plan`.

7. User Interface Documentation

These screenshots were supplied with the project request and document the implemented browser flow.

7.1 Profile / plan creation



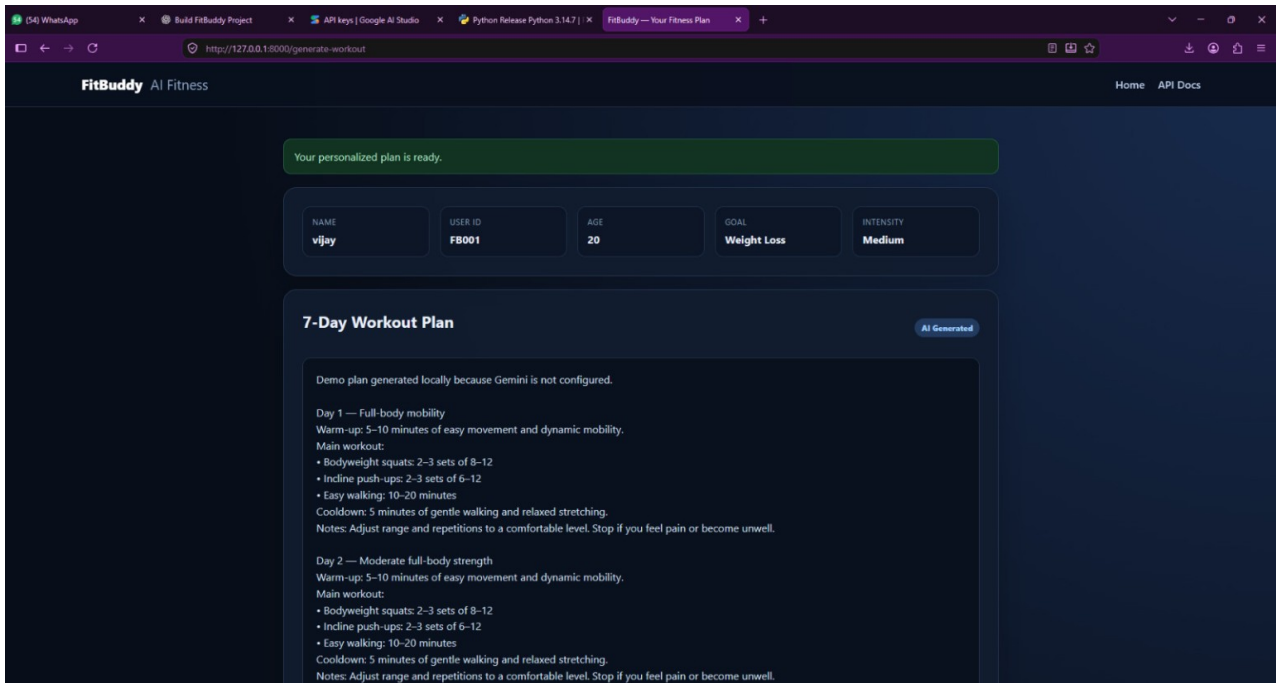
The screenshot shows the FitBuddy AI Fitness web application. The header includes the logo and navigation links for Home and API Docs. The main heading is "Build your personalized 7-day fitness plan." with a subtext: "Enter your fitness information and FitBuddy will create a structured workout plan with an AI-generated nutrition or recovery tip." Below this is a form titled "Your Fitness Profile" with the following fields:

- Name:
- User ID:
- Age:
- Weight (kg):
- Fitness Goal:
- Workout Intensity:
- Experience Level:

A "Generate My Plan" button is located at the bottom of the form.

Figure 1. Home screen with the fitness profile form and Generate My Plan action.

7.2 Generated plan



The screenshot shows the FitBuddy AI Fitness web application displaying the generated plan. The header is the same as Figure 1. A green notification bar at the top says "Your personalized plan is ready." Below this is a summary of the user's profile information:

NAME	USER ID	AGE	GOAL	INTENSITY
vijay	FB001	20	Weight Loss	Medium

Below the profile summary is the "7-Day Workout Plan" section, which includes a "AI Generated" badge. The plan content is as follows:

Demo plan generated locally because Gemini is not configured.

Day 1 — Full-body mobility
Warm-up: 5–10 minutes of easy movement and dynamic mobility.
Main workout:
• Bodyweight squats: 2–3 sets of 8–12
• Incline push-ups: 2–3 sets of 6–12
• Easy walking: 10–20 minutes
Cooldown: 5 minutes of gentle walking and relaxed stretching.
Notes: Adjust range and repetitions to a comfortable level. Stop if you feel pain or become unwell.

Day 2 — Moderate full-body strength
Warm-up: 5–10 minutes of easy movement and dynamic mobility.
Main workout:
• Bodyweight squats: 2–3 sets of 8–12
• Incline push-ups: 2–3 sets of 6–12
• Easy walking: 10–20 minutes
Cooldown: 5 minutes of gentle walking and relaxed stretching.
Notes: Adjust range and repetitions to a comfortable level. Stop if you feel pain or become unwell.

Figure 2. Generated-plan screen showing profile information and the seven-day workout output.

7.3 Feedback revision

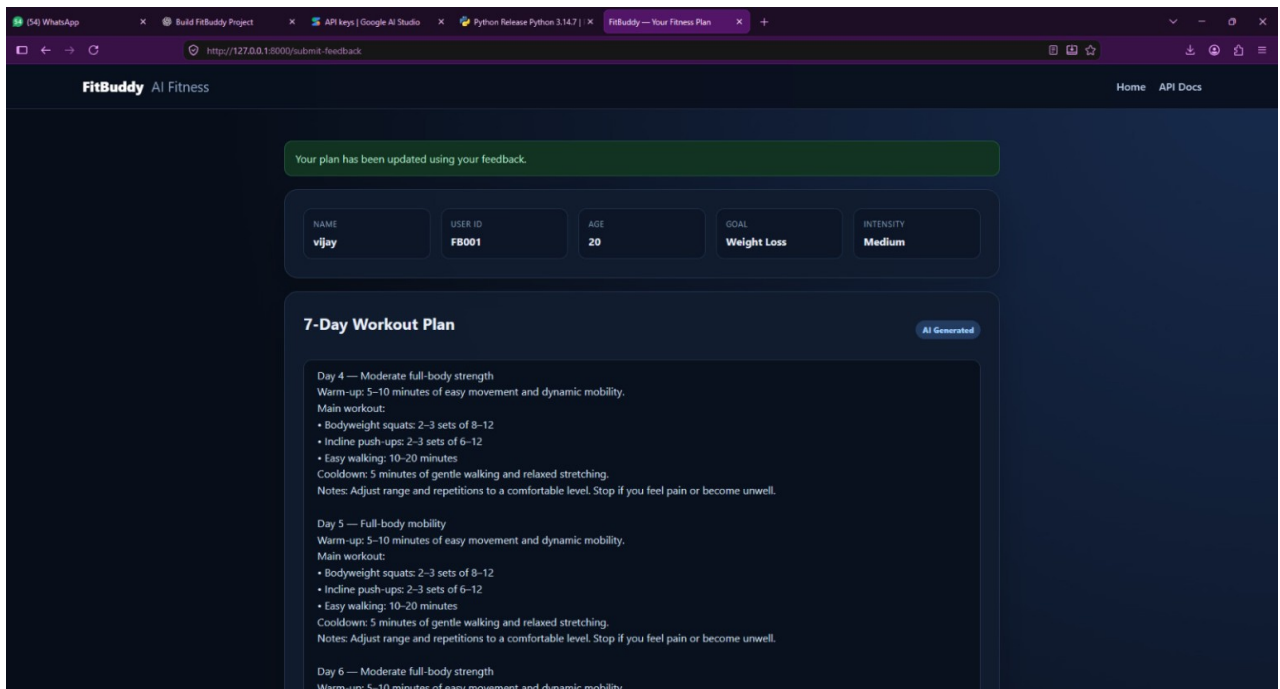


Figure 3. Updated-plan screen after feedback submission.

8. AI Service and Prompt Design

Client behavior

`get_client()` creates a Google GenAI client only when `GEMINI_API_KEY` is configured. Without a key, `_generate()` uses `mock_response()` when `ALLOW MOCK AI` is true; otherwise `AIServiceError` is raised.

Workout JSON contract

```
{
  "overview": "string",
  "days": [
    {
      "day": "Day 1",
      "focus": "string",
      "warm_up": "string",
      "main_workout": ["string"],
      "cool_down": "string",
      "notes": "string"
    }
  ]
}
```

Exactly 7 day objects are required.

Safety instructions embedded in prompts

- Adapt difficulty to intensity and experience.
- Include recovery/rest where appropriate.
- Include warm-up and cool-down guidance.
- Use accessible exercises and alternatives.
- Do not prescribe starvation, dehydration, unsafe challenges, extreme training volume, or medical treatment.
- If injury or a health condition is mentioned, recommend professional guidance.

Formatting

`pretty_plan()` converts JSON into readable text. If JSON parsing fails, the raw response is returned rather than raising a formatting exception. The mock response is deterministic and intended for demo/testing, not as a personalized production recommendation.

9. Database and Persistence

User table

Field	Type	Purpose
id	Integer PK	Internal identifier
user_id	String(80), unique/indexed	Application user ID
name	String(120)	User name
age	Integer	Age
weight	Float	Weight in kg
goal	String(80)	Fitness goal
intensity	String(20)	Workout intensity
experience_level	String(40)	Experience level
created_at / updated_at	DateTime	Record timestamps

Plan table

Field	Type	Purpose
id	Integer PK	Plan identifier
user_id	FK → users.user_id	Owner
original_plan	Text	Original generated content
updated_plan	Text nullable	Latest feedback revision
nutrition_tip	Text	Nutrition/recovery tip
feedback	Text nullable	Latest revision feedback
is_current	Boolean	Active plan marker
created_at / updated_at	DateTime	Plan timestamps

Version behavior

`save_plan()` marks older plans for the user as `is_current=false` and inserts the new plan as current. Feedback updates the current row, preserving the original plan and latest revision in the same record.

10. REST API Reference

Method	Path	Purpose	Success
GET	/api/v1/health	Configuration and health check	200
POST	/api/v1/plans	Create/update user and generate plan	200
GET	/api/v1/plans/{user_id}	Retrieve current plan	200
POST	/api/v1/plans/{user_id}/feedback	Revise current plan	200

POST /api/v1/plans

```
{
  "user_id": "FB001",
  "name": "Demo User",
  "age": 25,
  "weight": 70,
  "goal": "general wellness",
  "intensity": "medium",
  "experience_level": "beginner"
}
```

Returns user/profile fields plus workout_plan and nutrition_tip.

GET /api/v1/plans/{user_id}

Returns user data, original_plan, updated_plan, nutrition_tip, and feedback. Unknown users/plans produce HTTP 404.

POST /api/v1/plans/{user_id}/feedback

```
{"user_id": "FB001", "feedback": "Make the routine easier and add more recovery."}
```

The path and payload user IDs must match or HTTP 400 is returned. AI failures are mapped to HTTP 502.

Interactive docs

FastAPI automatically exposes Swagger UI at **/docs** and ReDoc at **/redoc**.

11. Configuration

Variable	Default in supplied source	Purpose
GEMINI_API_KEY	empty	Gemini credential
WORKOUT_MODEL	gemini-3.8-flash	Workout/revision model ID
NUTRITION_MODEL	gemini-3.8-flash	Nutrition model ID
DATABASE_URL	sqlite:///./fitbuddy.db	Database URL
ENVIRONMENT	development	Environment label
ALLOW MOCK_AI	true	Enable local fallback
ADMIN_USERNAME	admin	Admin Basic Auth username
ADMIN_PASSWORD	change-me	Admin Basic Auth password
MAX_FEEDBACK_LENGTH	1200	Feedback limit

.env example

```
# Copy this file to .env
# Get a Gemini API key from Google AI Studio.
GEMINI_API_KEY=

# Current Gemini model IDs. The source document names Gemini 1.5 Pro/Flash;
# these defaults use current Gemini API models so the project remains runnable.
WORKOUT_MODEL=gemini-3.8-flash
NUTRITION_MODEL=gemini-3.8-flash

DATABASE_URL=sqlite:///./fitbuddy.db
ENVIRONMENT=development

# Local demo fallback. Set false to require Gemini.
ALLOW MOCK_AI=true

# Admin dashboard credentials. Change these before any non-local deployment.
ADMIN_USERNAME=admin
ADMIN_PASSWORD=change-me
MAX_FEEDBACK_LENGTH=1200
```

Important configuration note

The supplied README says the original specification referenced historical Gemini model names, while the implementation uses configurable model IDs. The identifiers above are copied from the supplied source and should be changed if the target AI project exposes different models.

12. Installation and Running

Prerequisites

- Python 3.11+ according to the README.
- Terminal/IDE and project directory.
- For live AI: Gemini API key and valid model IDs.
- For demo/testing: mock mode can run without an AI key.

Windows PowerShell

```
python -m venv .venv
.\\.venv\\Scripts\\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
```

macOS / Linux

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -r requirements.txt
```

Run

```
uvicorn app.main:app --reload
```

Open <http://127.0.0.1:8000>. Swagger: </docs>. ReDoc: </redoc>.

Test

```
pytest -q
```

13. Testing and Quality Assurance

The supplied tests use FastAPI TestClient and mock AI, so they do not depend on a live Gemini service.

Test	Verification
test_home	GET / returns 200 and includes FitBuddy.
test_health	Health endpoint returns status=ok.
test_create_plan_with_mock_ai	Plan creation returns user ID, Day 1, and nutrition tip.
test_feedback_with_mock_ai	Feedback revision succeeds and keeps Day 7.

Recommended additional coverage

- Invalid goal/intensity/experience values.
- Age and weight boundary tests.
- 404 behavior for unknown users/plans.
- Feedback length > 1,200.
- Mismatched path/payload user IDs.
- Gemini empty-response and exception paths.
- Admin authentication failures.
- Plan-current flag behavior when generating a second plan.

14. Security and Safety

Authentication

The admin dashboard is protected with HTTP Basic Auth and `secrets.compare_digest()`. The supplied default password is **change-me**; it must be replaced before non-local use.

Input and application security

- Pydantic validates lengths, ranges, and enumerations.
- Feedback is limited to 1,200 characters.
- Jinja2 templates are used for HTML rendering.
- SQLAlchemy ORM is used for database access rather than string-built SQL.

AI safety

- Workout prompts prohibit starvation, dehydration, unsafe challenges, extreme volume, and medical treatment.
- Nutrition prompts prohibit restrictive dieting, calorie targets, supplements, and medical treatment.
- The UI/README frame the output as general wellness content, not medical diagnosis or treatment.

Production hardening

- Use HTTPS.
- Replace Basic Auth with stronger identity/session management.
- Keep secrets outside source control.
- Add CSRF protection for authenticated browser forms.
- Add rate limiting around AI endpoints.
- Add structured logging, monitoring, and privacy controls.
- Validate AI responses against a strict Pydantic output schema.

15. Troubleshooting

Symptom	Likely cause	Action
GEMINI_API_KEY not configured	No key and mock disabled	Add key or enable mock mode for local use.
Gemini request failed	Credential, quota, network, or model issue	Check key, model ID, connectivity, and provider availability.
Database issues	Local SQLite state	Stop server, remove fitbuddy.db, restart.
PowerShell activation blocked	Local execution policy	Use approved alternative activation method.
Admin shows no users	No records in active database	Create a plan and verify DATABASE_URL.

Clean local reset

```
# Stop Uvicorn first
rm fitbuddy.db          # macOS/Linux
del fitbuddy.db         # Windows Command Prompt
uvicorn app.main:app --reload
```

16. Limitations and Future Work

Current limitations

- SQLite is a local/demo-oriented persistence choice.
- Admin authentication is Basic Auth.
- No database migration framework is included.
- AI workout output is not validated by a dedicated Pydantic response schema.
- Mock AI is deterministic and not genuinely personalized.
- No explicit rate limiting, CSRF protection, audit trail, or production observability is included.
- AI provider model availability can change over time.

Future enhancements

- Validate generated JSON with Pydantic models.
- Add full plan history/versioning.
- Introduce stronger user authentication and roles.
- Use Alembic and PostgreSQL for long-lived deployment.
- Add rate limiting, request IDs, structured logging, metrics, and readiness checks.
- Add equipment, duration, substitutions, muscle-group, and difficulty metadata.
- Support configurable training days and available equipment.
- Add printable/PDF workout exports.
- Add UI and AI-output regression tests.
- Add human review for unusual or safety-sensitive profiles.

17. Deployment Checklist

- Replace default admin password.
- Supply Gemini credentials through a secure secret mechanism.
- Confirm workout/nutrition model IDs are available.
- Set ALLOW MOCK AI=false where silent fallback is undesirable.
- Use a production database.
- Run the complete test suite.
- Use HTTPS.
- Add rate limiting and appropriate authentication.
- Review privacy requirements for stored profile data.
- Monitor AI errors, latency, quotas, and output quality.
- Back up the database and define recovery procedures.
- Review AI safety prompts whenever product scope changes.

18. Appendix: Key Source Reference

Application bootstrap

```
from contextlib import asynccontextmanager
from fastapi import FastAPI
from fastapi.staticfiles import StaticFiles
from .config import BASE_DIR, get_settings
from .database import init_db
from .routes import router, api

@asynccontextmanager
async def lifespan(app: FastAPI):
    init_db()
    yield

settings = get_settings()
app = FastAPI(
    title=settings.app_name,
    description="AI-powered 7-day fitness planning demo based on the supplied FitBuddy project specification.",
    version="1.0.0",
    lifespan=lifespan,
)

app.mount("/static", StaticFiles(directory=str(BASE_DIR / "static")), name="static")
app.include_router(router)
app.include_router(api)
```

Validation models

```
from pydantic import BaseModel, Field, field_validator

ALLOWED_GOALS = {"weight loss", "muscle gain", "general wellness", "flexibility", "endurance"}
ALLOWED_INTENSITIES = {"low", "medium", "high"}
ALLOWED_EXPERIENCE = {"beginner", "intermediate", "advanced"}

class UserInput(BaseModel):
    user_id: str = Field(min_length=2, max_length=80)
    name: str = Field(min_length=1, max_length=120)
    age: int = Field(ge=13, le=100)
    weight: float = Field(gt=20, lt=400)
    goal: str = Field(min_length=2, max_length=80)
    intensity: str = Field(min_length=3, max_length=20)
    experience_level: str = Field(default="beginner", max_length=40)

    @field_validator("user_id", "name", "goal", "intensity", "experience_level")
    @classmethod
    def strip_values(cls, value: str) -> str:
        return value.strip()

    @field_validator("goal")
    @classmethod
    def valid_goal(cls, value: str) -> str:
        normalized = value.lower()
        if normalized not in ALLOWED_GOALS:
            raise ValueError(f"Goal must be one of: {' '.join(sorted(ALLOWED_GOALS))}")
        return normalized

    @field_validator("intensity")
    @classmethod
    def valid_intensity(cls, value: str) -> str:
        normalized = value.lower()
        if normalized not in ALLOWED_INTENSITIES:
            raise ValueError("Intensity must be low, medium, or high")
        return normalized

    @field_validator("experience_level")
    @classmethod
    def valid_experience(cls, value: str) -> str:
        normalized = value.lower()
        if normalized not in ALLOWED_EXPERIENCE:
            raise ValueError("Experience must be beginner, intermediate, or advanced")
        return normalized

class FeedbackRequest(BaseModel):
    user_id: str = Field(min_length=2, max_length=80)
    feedback: str = Field(min_length=3, max_length=1200)

class PlanResponse(BaseModel):
```

```

user_id: str
name: str
goal: str
intensity: str
experience_level: str
workout_plan: str
nutrition_tip: str

```

AI service

```

import json
import re
from functools import lru_cache
from .config import get_settings

class AIServiceError(RuntimeError):
    pass

@lru_cache
def get_client():
    settings = get_settings()
    if not settings.gemini_api_key:
        return None
    from google import genai
    return genai.Client(api_key=settings.gemini_api_key)

def _clean_text(text: str) -> str:
    text = (text or "").strip()
    text = re.sub(r"^\`(?:json|text)?\s*", "", text, flags=re.I)
    text = re.sub(r"\s*\`$", "", text)
    return text.strip()

def _generate(model: str, prompt: str, *, temperature: float = 0.6, json_output: bool = False) -> str:
    settings = get_settings()
    client = get_client()
    if client is None:
        if settings.allow_mock_ai:
            return mock_response(prompt)
        raise AIServiceError("GEMINI_API_KEY is not configured.")
    try:
        from google.genai import types
        config = types.GenerateContentConfig(
            temperature=temperature,
            max_output_tokens=7000,
            response_mime_type="application/json" if json_output else None,
        )
        response = client.models.generate_content(model=model, contents=prompt, config=config)
        text = _clean_text(response.text)
        if not text:
            raise AIServiceError("Gemini returned an empty response.")
        return text
    except Exception as exc:
        raise AIServiceError(f"Gemini request failed: {exc}") from exc

def _profile(user) -> str:
    return (
        f"Name: {user.name}\nAge: {user.age}\nWeight: {user.weight} kg\n"
        f"Goal: {user.goal}\nIntensity: {user.intensity}\n"
        f"Experience: {user.experience_level}\n"
    )

def generate_workout_gemini(user) -> str:
    settings = get_settings()
    prompt = f"""You are FitBuddy, a cautious fitness-planning assistant.
    Create a safe, practical 7-day fitness plan for the following profile.

    {_profile(user)}

    Return JSON with exactly these keys:
    "overview": string,
    "days": [
        {{ "day": "Day 1", "focus": string, "warm_up": string,
          "main_workout": [string], "cool_down": string, "notes": string }}
    ]
    The days array must contain exactly 7 objects.

    Requirements:
    - Adapt difficulty to intensity and experience.
    - Include rest/recovery where appropriate.
    - Include warm-up and cool-down guidance.
    - Use accessible exercises and alternatives.

```

- Do not prescribe starvation, dehydration, unsafe challenges, extreme training volume, or medical treatment.
- Avoid medical diagnoses. If a health condition or injury is mentioned, recommend professional guidance.
- Keep the plan easy to read and actionable.

"""

```
    return _generate(settings.workout_model, prompt, temperature=0.55, json_output=True)
```

```
def generate_nutrition_tip_with_flash(user) -> str:
```

```
    settings = get_settings()
```

```
    prompt = f"""You are FitBuddy's nutrition and recovery assistant.
```

```
    Give one concise, general wellness tip for this profile:
```

```
{_profile(user)}
```

The response must be 80 words or fewer. Focus on balanced meals, hydration, sleep, recovery, or a sensible source of protein/fiber depending on the goal.

Do not prescribe restrictive dieting, calorie targets, supplements, or medical treatment.

"""

```
    return _generate(settings.nutrition_model, prompt, temperature=0.4)
```

```
def update_workout_plan(user, original_plan: str, feedback: str) -> str:
```

```
    settings = get_settings()
```

```
    prompt = f"""You are FitBuddy. Revise the original 7-day workout plan using the user's feedback.
```

```
USER:
```

```
{_profile(user)}
```

```
ORIGINAL PLAN:
```

```
{original_plan}
```

```
FEEDBACK:
```

```
{feedback}
```

Return JSON with exactly:

```
"overview": string,
```

```
"days": [
```

```
    {"day": "Day 1", "focus": string, "warm_up": string,
```

```
     "main_workout": [string], "cool_down": string, "notes": string}]
```

```
]
```

Keep exactly 7 days. Preserve useful parts of the original plan, incorporate reasonable feedback, and keep the plan safe. Do not create extreme exercise, unsafe challenges, starvation/dehydration, or medical treatment. Include recovery/rest when appropriate.

"""

```
    return _generate(settings.workout_model, prompt, temperature=0.55, json_output=True)
```

```
def pretty_plan(plan_text: str) -> str:
```

```
    try:
```

```
        data = json.loads(plan_text)
```

```
        lines = [data.get("overview", "Personalized 7-day plan"), ""]
```

```
        for day in data.get("days", []):
```

```
            lines.append(f"{day.get('day', '')} – {day.get('focus', '')}")
```

```
            lines.append(f"Warm-up: {day.get('warm_up', '')}")
```

```
            lines.append("Main workout:")
```

```
            for item in day.get("main_workout", []):
```

```
                lines.append(f"• {item}")
```

```
            lines.append(f"Cooldown: {day.get('cool_down', '')}")
```

```
            if day.get("notes"):
```

```
                lines.append(f"Notes: {day['notes']}")
```

```
            lines.append("")
```

```
        return "\n".join(lines).strip()
```

```
    except (json.JSONDecodeError, TypeError):
```

```
        return plan_text
```

```
def mock_response(prompt: str) -> str:
```

```
    if "days array must contain exactly 7 objects" in prompt or "Keep exactly 7 days" in prompt:
```

```
        days = []
```

```
        for i in range(1, 8):
```

```
            focus = "Full-body mobility" if i in (1, 5) else ("Recovery / rest" if i == 7 else "Moderate full-body strength")
```

```
            days.append({
```

```
                "day": f"Day {i}",
```

```
                "focus": focus,
```

```
                "warm_up": "5-10 minutes of easy movement and dynamic mobility.",
```

```
                "main_workout": ["Bodyweight squats: 2-3 sets of 8-12", "Incline push-ups: 2-3 sets of 6-12", "Easy walking"],
```

```
                "cool_down": "5 minutes of gentle walking and relaxed stretching.",
```

```
                "notes": "Adjust range and repetitions to a comfortable level; stop if you feel pain or unwell."
```

Database models and helpers

```
from datetime import datetime, timezone
```

```
from sqlalchemy import Boolean, DateTime, Float, ForeignKey, Integer, String, Text, create_engine
```

```
from sqlalchemy.orm import DeclarativeBase, Mapped, Session, mapped_column, relationship, sessionmaker
```

```
from .config import get_settings
```

```

settings = get_settings()
connect_args = {"check_same_thread": False} if settings.database_url.startswith("sqlite") else {}
engine = create_engine(settings.database_url, connect_args=connect_args)
SessionLocal = sessionmaker(bind=engine, autoflush=False, autocommit=False, expire_on_commit=False)

def utcnow():
    return datetime.now(timezone.utc)

class Base(DeclarativeBase):
    pass

class User(Base):
    __tablename__ = "users"
    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    user_id: Mapped[str] = mapped_column(String(80), unique=True, index=True)
    name: Mapped[str] = mapped_column(String(120))
    age: Mapped[int] = mapped_column(Integer)
    weight: Mapped[float] = mapped_column(Float)
    goal: Mapped[str] = mapped_column(String(80))
    intensity: Mapped[str] = mapped_column(String(20))
    experience_level: Mapped[str] = mapped_column(String(40), default="beginner")
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utcnow)
    updated_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utcnow, onupdate=utcnow)
    plans: Mapped[list["Plan"]] = relationship(back_populates="user", cascade="all, delete-orphan")

class Plan(Base):
    __tablename__ = "plans"
    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    user_id: Mapped[str] = mapped_column(ForeignKey("users.user_id", ondelete="CASCADE"), index=True)
    original_plan: Mapped[str] = mapped_column(Text)
    updated_plan: Mapped[str | None] = mapped_column(Text, nullable=True)
    nutrition_tip: Mapped[str] = mapped_column(Text)
    feedback: Mapped[str | None] = mapped_column(Text, nullable=True)
    is_current: Mapped[bool] = mapped_column(Boolean, default=True)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utcnow)
    updated_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utcnow, onupdate=utcnow)
    user: Mapped[User] = relationship(back_populates="plans")

def init_db():
    Base.metadata.create_all(bind=engine)

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

def save_user(db: Session, data):
    user = db.query(User).filter(User.user_id == data.user_id).first()
    if user:
        for field in ("name", "age", "weight", "goal", "intensity", "experience_level"):
            setattr(user, field, getattr(data, field))
    else:
        user = User(**data.model_dump())
        db.add(user)
    db.commit()
    db.refresh(user)
    return user

def save_plan(db: Session, user_id: str, original_plan: str, nutrition_tip: str):
    db.query(Plan).filter(Plan.user_id == user_id).update({"is_current": False})
    plan = Plan(user_id=user_id, original_plan=original_plan, nutrition_tip=nutrition_tip)
    db.add(plan)
    db.commit()
    db.refresh(plan)
    return plan

def get_current_plan(db: Session, user_id: str):
    return (
        db.query(Plan)
        .filter(Plan.user_id == user_id, Plan.is_current.is_(True))
        .order_by(Plan.id.desc())
        .first()
    )

def update_plan(db: Session, plan: Plan, updated_plan: str, feedback: str, nutrition_tip: str | None = None):
    plan.updated_plan = updated_plan
    plan.feedback = feedback
    if nutrition_tip:
        plan.nutrition_tip = nutrition_tip
    db.commit()

```

```
        db.refresh(plan)
        return plan

def get_all_users(db: Session):
    return db.query(User).order_by(User.created_at.desc()).all()

def get_all_plans(db: Session):
    return db.query(Plan).order_by(Plan.created_at.desc()).all()

def get_user(db: Session, user_id: str):
    return db.query(User).filter(User.user_id == user_id).first()
```

End of Documentation

FitBuddy – AI Fitness Plan Generator

Prepared from the supplied source archive and screenshots.